

Fun With Darts
Ricky and Vivaan

Why are darts hard?

Variable	Typical Value	Effect of Small Error
Release speed	5–7 m/s	± 0.1 m/s shifts landing ≈ 10 – 15 mm vertically
Vertical launch angle	15–25° above horizontal	$\pm 1^\circ$ shifts landing ≈ 40 mm vertically
Horizontal launch angle	$\approx 0^\circ$ (straight ahead)	$\pm 0.5^\circ$ shifts landing ≈ 20 mm laterally
Release timing	~ 0 ms relative to optimal	± 5 ms changes both speed and angle
Spin rate	0–10 rev/s	Affects in-flight stability and impact angle
Dart weight	18–26 g	Heavier darts less sensitive to aerodynamic perturbations

How are humans at darts?

- The best player averages ~100 per visit
- Humans are inherently noisy due to neuromuscular transmission (“motor noise”)
- Adapted professional techniques:
 - Elbow-driven throws
 - Wrist snap
 - Planar motion
 - Stable stance

The goal

We want a robot arm that throws darts at a board

Small changes (release timing, torque, wind) → different landing.

We don't want one perfect throw – we want a policy that gets high expected score even with noise.

Why RL?

Arm dynamics + release + flight are complex and stochastic.

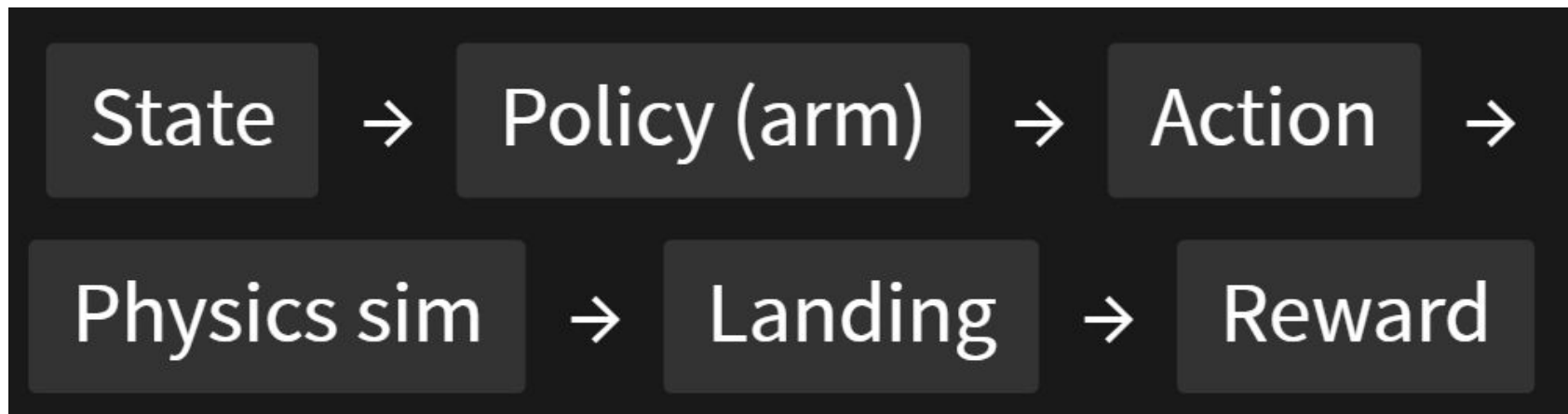
Analytical control (solve for exact torques) is intractable under noise

RL learns a mapping: state $\rightarrow$ action by trial and error in simulation.

The policy is our way of encoding “how the arm should work” for this task.

Validated on real hardware: Barrett WAM learned Around the Clock in $\sim 1,000$ throws (Kober et al.)

RL Loop



- **State:** joint angles, joint velocities, (optional) wind estimate.
- **Action:** joint torques (e.g. wrist), release timing.
- **Sim:** our 3D projectile + dartboard (what we already built).
- **Reward:** dartboard score (and maybe smoothness / stability penalties).

State space

- Arm: joint angles θ , joint angular velocities $\dot{\theta}$.
- Optional: estimated wind, or dart orientation at release.
- From state we can compute (or estimate) **release position & velocity** → then run our physics to get landing.

Action space

- **Continuous:** e.g. wrist torque, other joint torques over a short throw phase.
- **Release timing:** when to release the dart (discrete event in an otherwise continuous control).
- We train a policy (e.g. neural network) that outputs these actions given the current state

Reward

- **Primary:** dartboard score at landing (from our `score_at(x, y)`).
- Optional: penalize unstable release, large torques, or variance across rollouts.
- We can use **expected score** (Monte Carlo over many noisy rollouts) so the policy optimizes for robustness

How we made the simulator

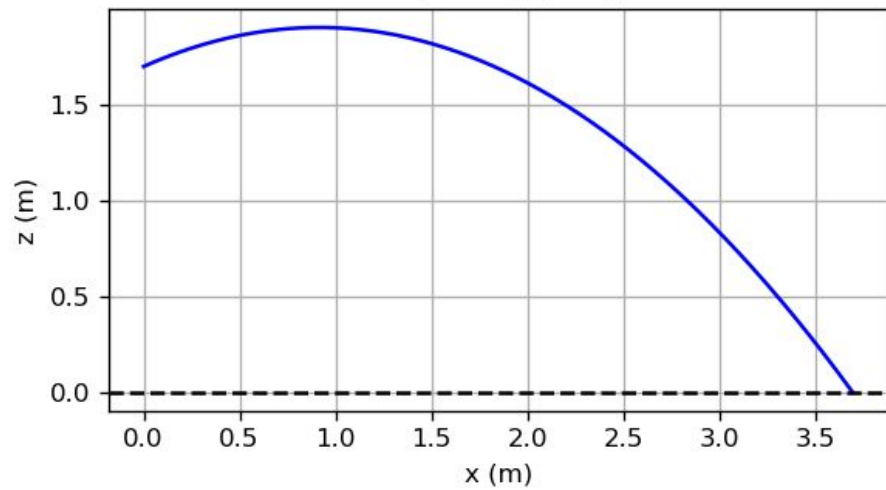
You pick a nominal release (same position and velocity every time).

You add random noise to that release (e.g. to velocity, and optionally wind) so each throw is slightly different.

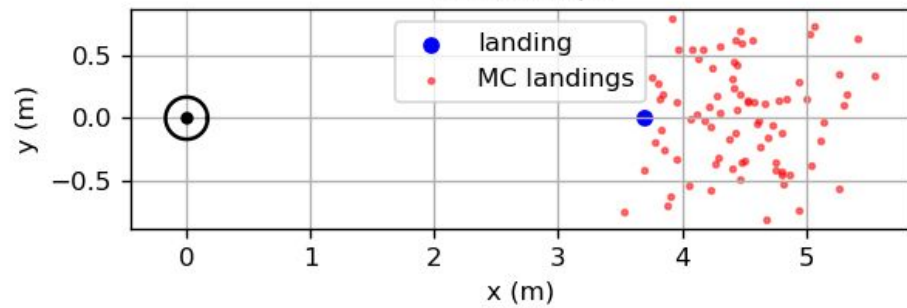
You run many such throws (e.g. 80 or 200) and record where each dart lands.

That set of landing points is the MC landings - a sample of possible outcomes.

Trajectory (side)



Board (top)



Results

3D throw: The demo throw with $\text{vel0} = [4.5, 0, 2]$ lands at 3.7 m, so it goes past the board (radius 0.17 m) and gets score 0. That's expected for that velocity.

Hit any number: T20, S19, D5, and BULL all land on the right segment and return the correct scores (20, 19, 5, 50).

Variance: Monte Carlo used a different nominal velocity that also overshoots, so mean landing is ~4.5 m and expected score is 0.

State estimator maps 2-link arm state $\rightarrow$ release pos/vel

Finding Robust Release Conditions

- Not all release states are equally fragile to noise
- We use sensitivity analysis to find noise-robust release conditions, then RL to generalize across the board
- This tells us: which release parameters does the landing error depend on most?
- Optimal release sits in the "flattest" region of the cost landscape where small perturbations cause minimal miss

Two problems, not just one

1. Direct sensitivity analysis (Jacobian computation) on projectile dynamics → find release conditions that minimize landing variance
2. iLQR on arm dynamics → find torques that achieve those release conditions

We decouple at the release state, training each side independently

Manipulator

1. Fixed Launcher (Cannon / Catapult)

- a. Pros: Extremely repeatable, easy to model, near-zero variance on a single target.
- b. Cons: No spin control, dart tumbles and bounces out. Can't retarget without full recalibration. Not competition-legal. A ballistics demo, not a control problem.

2. Rigid Arm + Simple Gripper (6-DOF arm, parallel-jaw release)

- a. Pros: Sub-millisecond timing, $<0.01^\circ$ encoder precision, fast retargeting via IK. Rich toolchain (ROS, MuJoCo).
- b. Cons: Parallel jaws let the dart slide out rather than throw. Spin and release angle couple to gripper geometry. Contact dynamics at release (friction, stiction, wobble) are discontinuous and hard to sim.

3. Dexterous Hand (3–5 finger anthropomorphic)

- a. Pros: Closest to human mechanics, where fingers can impart spin, shape release attitude, and learn stabilizing techniques. Most generalizable
- b. Cons: Huge action space (every finger joint is another DOF). Multi-fingertip contact on a thin barrel is a sim-to-real nightmare. Hardware is expensive, fragile, and adds mechanical noise sources.

Our Aim

We don't hand-design torques or timings.

We define state, action, reward and let RL (e.g. SAC, TD3, PPO) learn a policy.

The learned policy = “given this arm state (and env), do this action.”

So the arm's behavior is discovered by the algorithm using our physics + dartboard as the environment.

Training pipeline

1. Build an **RL environment**: $\text{step}(\text{state}, \text{action}) \rightarrow$ run arm dynamics to release, then our projectile + dartboard to get landing and score.
2. Use a standard **off-the-shelf RL algorithm** (e.g. Stable-Baselines3) to train the policy.
3. Optionally add **curriculum**: start with little noise, then increase wind/release variance.
4. Evaluate with **Monte Carlo**: many rollouts per policy $\rightarrow$ distribution of landings and expected score.

Summary

- RL learns a **policy** that maps arm (and env) state to torques and release timing.
- Our **physics simulator** provides the transition: action → release → trajectory → landing → score.
- By maximizing expected score under noise, we get a robust “recipe” for how the arm should move — that’s what we mean by figuring out how the arm works with RL.